

From Vibe Coding to Spec-Driven Agentic Engineering

A Hands-On Implementation using Claude Code

PART X

Instructor: Muhammad Arif Butt, PhD

About The Instructor



Dr. Muhammad Arif Butt
Asst. Prof. Department of Data Science
University of Punjab, Lahore
arif@pucit.edu.pk
https://arif.excaliat@gmail.com
https://arifbutt.me
https://www.linkedin.com/in/dr-arif-butt/
https://youtube.com/learnwitharif
https://github.com/arifpucit
https://huggingface.co/arif-butt

- **Education:**
  - Graduation from Pakistan Military Academy, Kakul
  - MPhil CS from University of Punjab, Lahore
  - PhD CS from University of Punjab, Lahore
- **Experience:**
  - Served in field/staff/instructional posts in Pakistan Army
  - Assistant Professor, Department of Data Science
  - Founder Excaliat (https://excaliat.com/en)
  - Founder FalconHunt (https://falconhunt.org/)
  - Co-Founder Tbox Solutionz (https://tboxsolutionz.com/)
- **Research Interest:**
  - Embedded and Real-Time Operating Systems
  - Vulnerability Analysis, Binary Exploitation & Exploit Development
  - AI-Driven Cybersecurity and Securing AI systems

Scheme of Sessions

Part - I: Overview of Agentic Coding Assistants

Part – II: Claude Ecosystem, Installing & using Claude Code

Part – III: Slash Commands, Context Window & `.claude` Directory

Part – IV: Workflow of Building a Basic Vulnerable Web Application

Part – V: Exploiting Vulnerable Web Application

Part – VI: Workflow of Implementing Features and Fixing Bugs

Part – VII: Collaborative Workflow: Claude Code + GitHub + Jira

Part – VIII: Built-in Skills & Vuln-Web-App Current Status

Part – IX: Creating & using Custom Skills & Skill Marketplace

Part – X: **Deep Dive: Conceptual Understanding of MCP**

Part – XI: MCP in Action

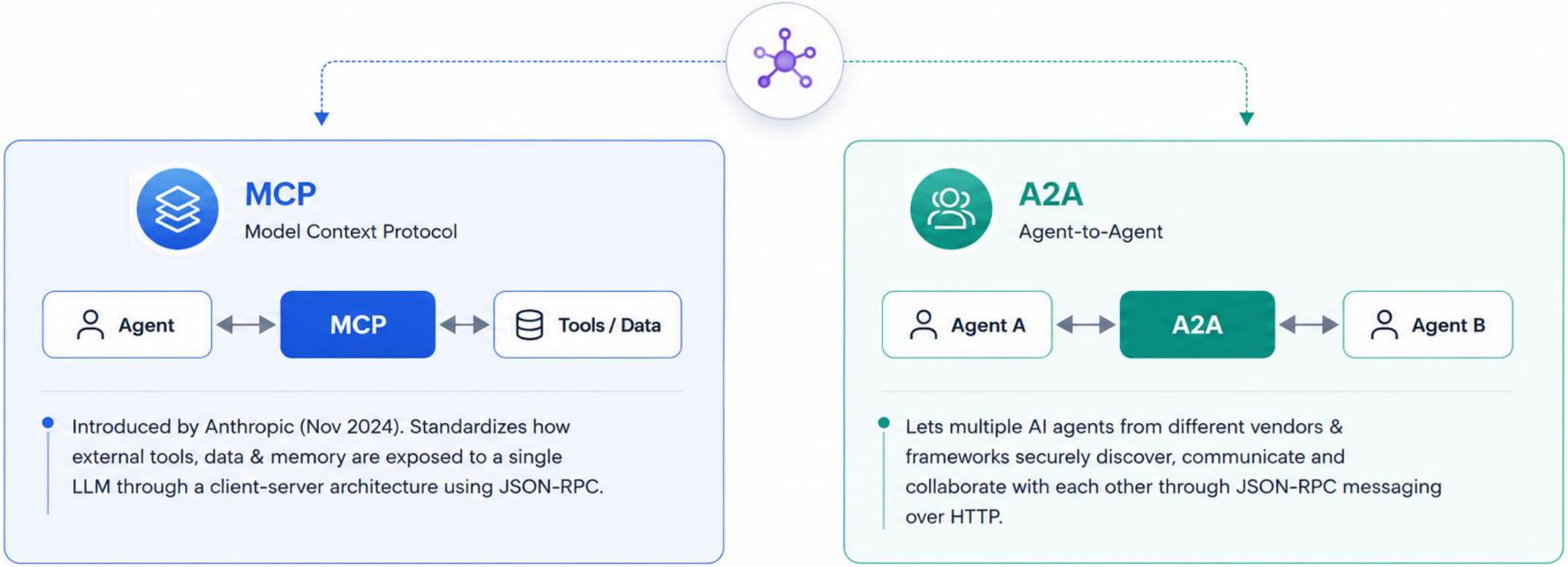
Important Concepts

Model Context Protocol

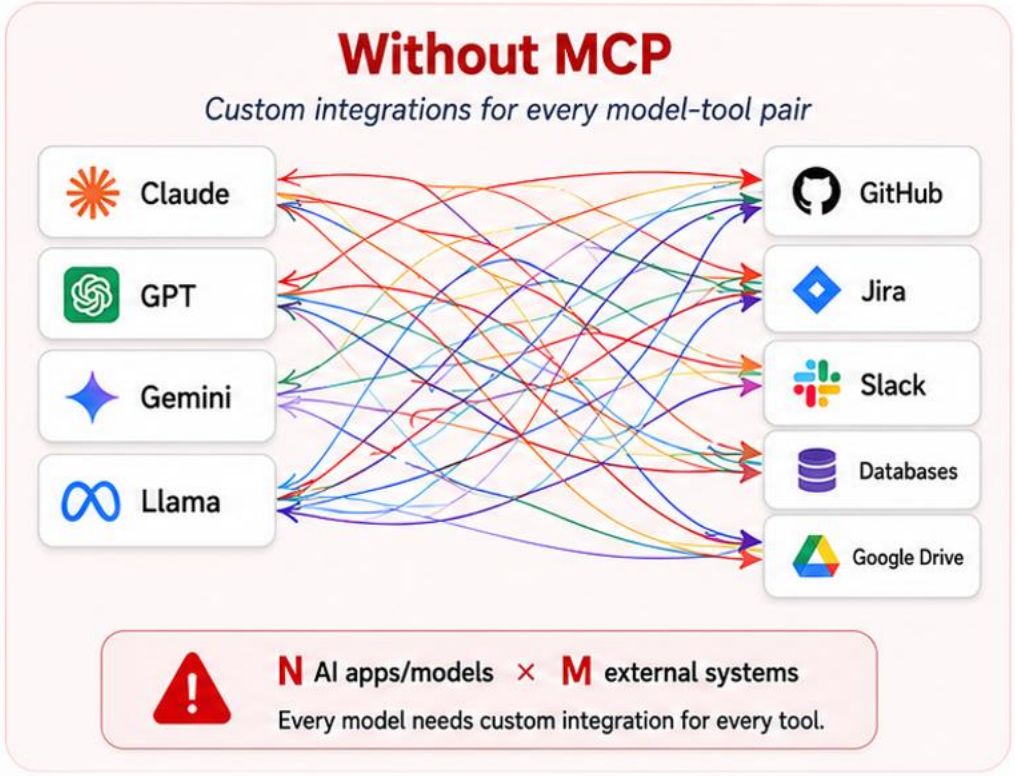
Instructor: Muhammad Arif Butt, PhD

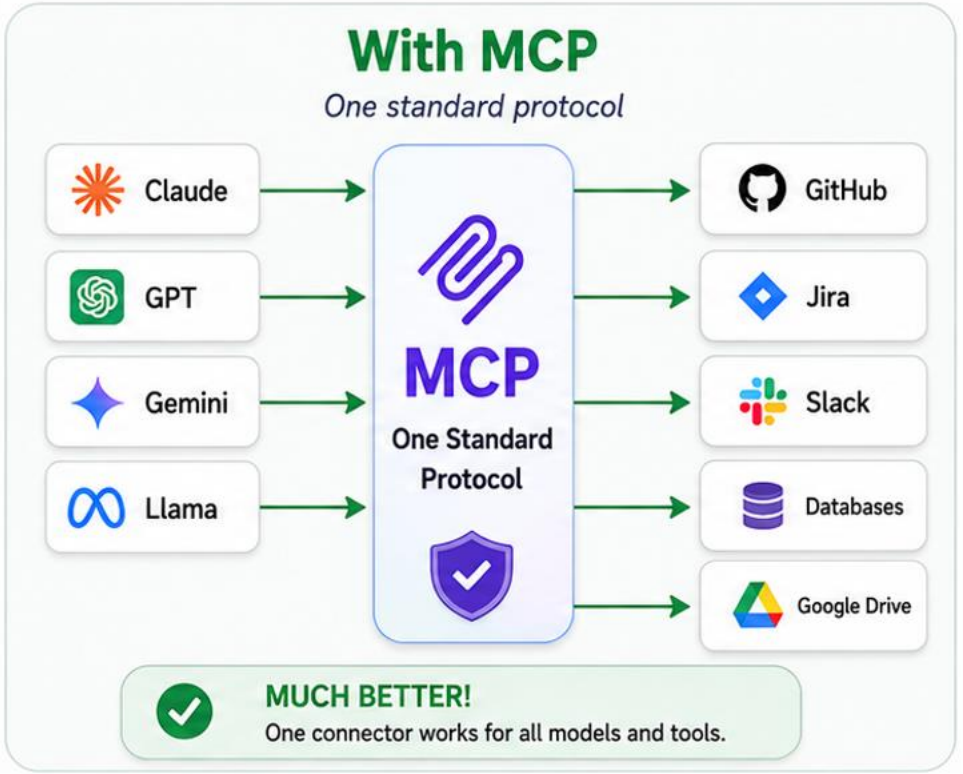
Agentic Protocols

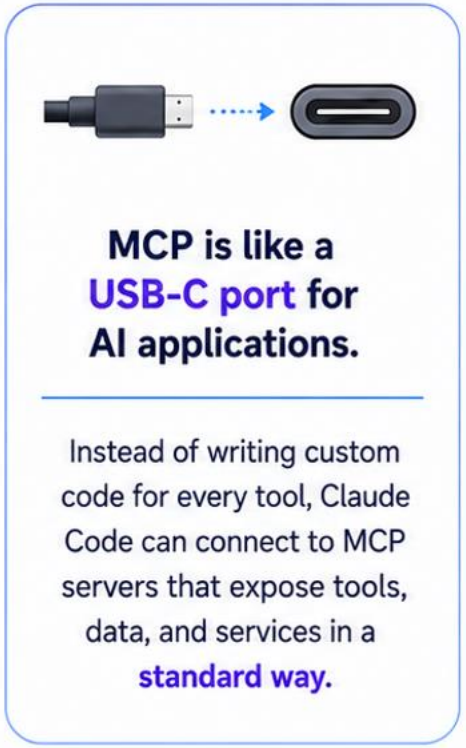
Standardized communication & control mechanisms that let LLMs, tools, memory systems and other agents interact reliably, safely & predictably.



MCP Solves the Tool Integration Problem







MCP | **MCP standardizes how AI agents connect to external systems.**

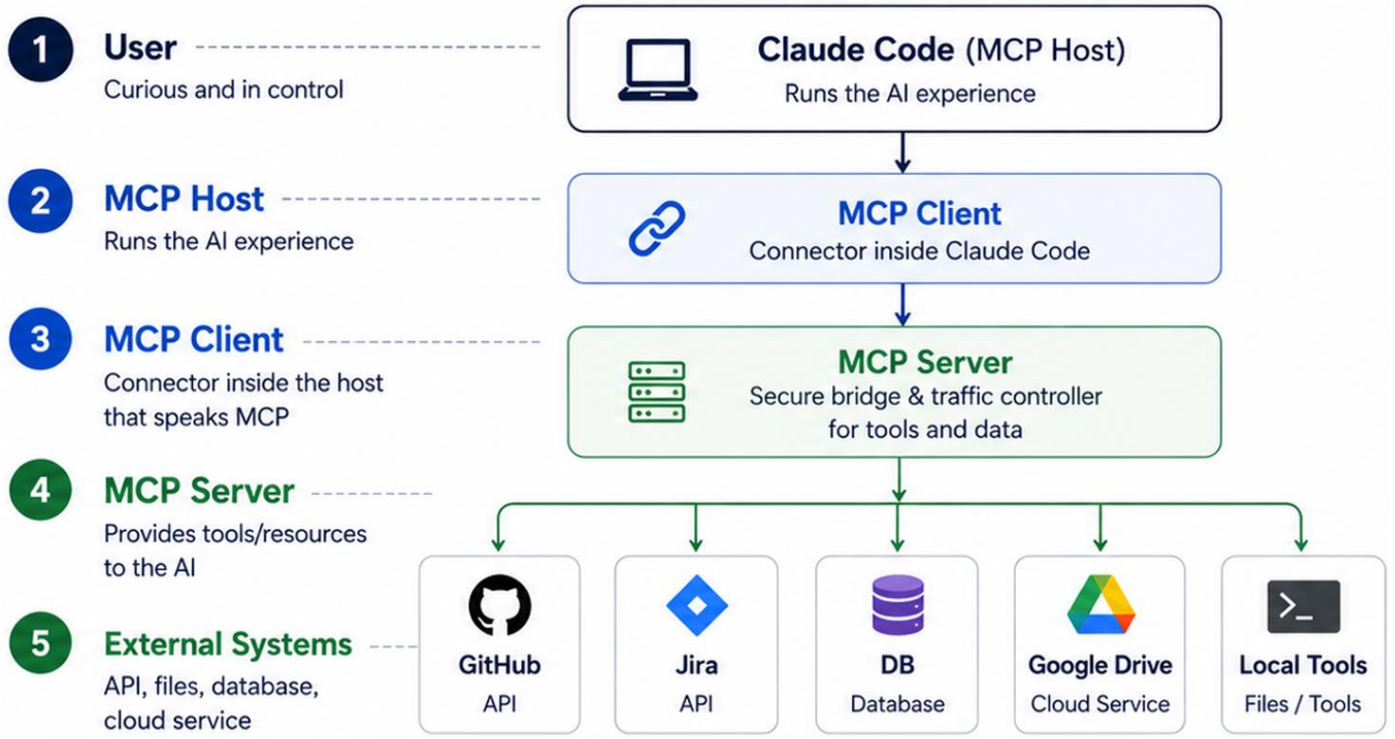
Claude: https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview

GPT: https://developers.openai.com/api/docs/guides/function-calling

Gemini: https://ai.google.dev/gemini-api/docs/function-calling

MCP Architecture

MCP (Model Context Protocol) is an open standard created by Anthropic that acts as a universal connector using which an agent can connect with external tools, services, and data sources.



What MCP Enables

Discovery
Claude discovers available tools, resources, and prompts from the MCP server.

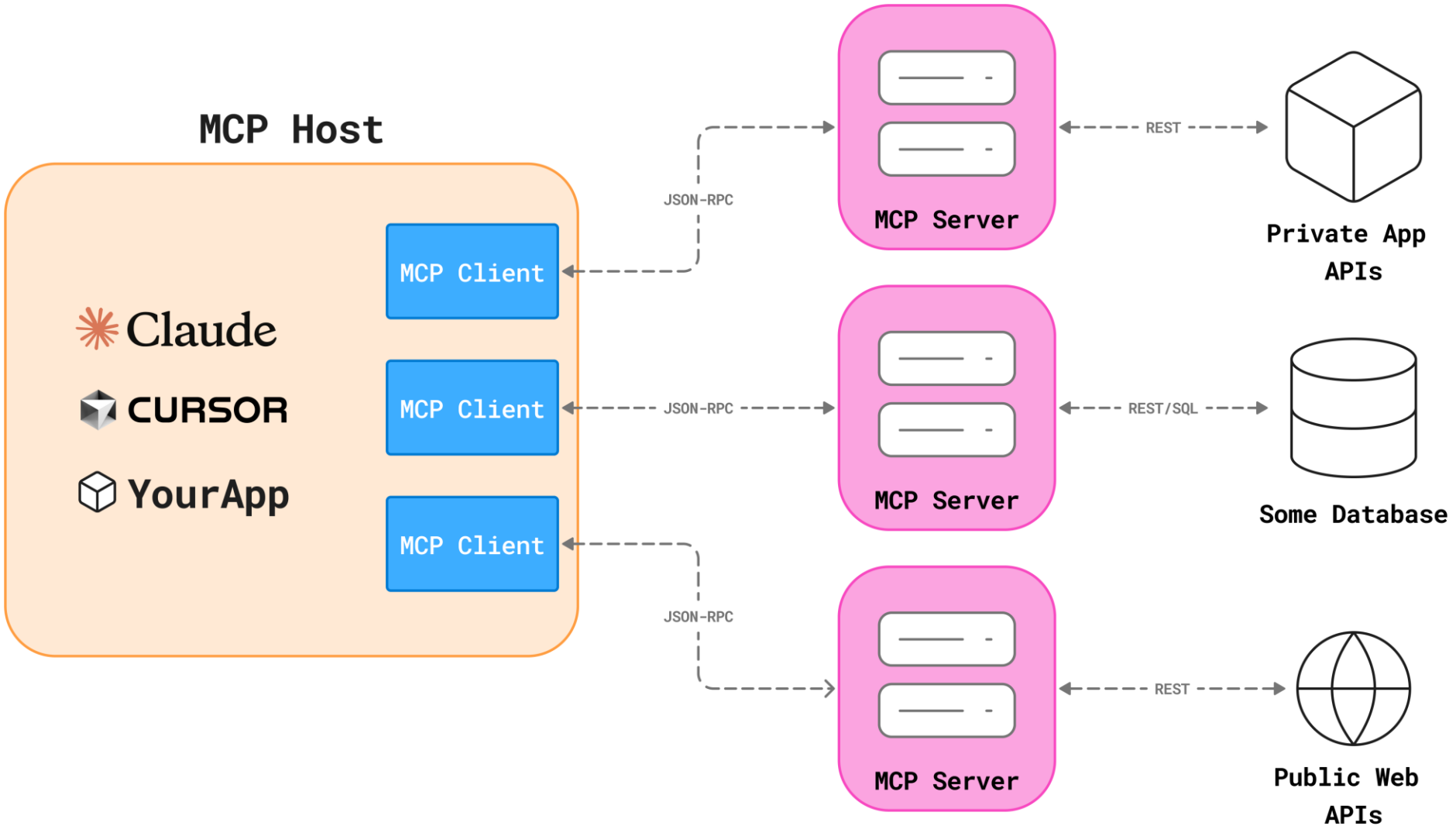
Standardized Access
A consistent protocol for AI models to connect to external capabilities.

Secure by Design
Connections and permissions are controlled by the server.

Extensible
Anyone can build MCP servers and expose their capabilities.

More Powerful AI
Claude can take action, read data, and complete real-world tasks using your systems.

MCP Communication

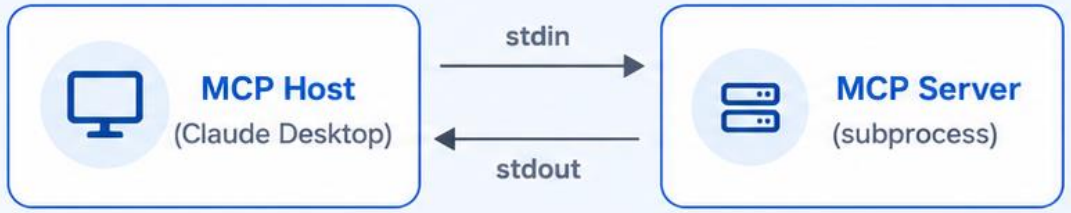


MCP Transport: Local vs Remote

Local MCP Server

stdio transport

Runs on your machine as a subprocess



PROS

- ✓ Very fast direct inter-process communication
- ✓ Secure – no open network port
- ✓ Required for local file / desktop access

CONS

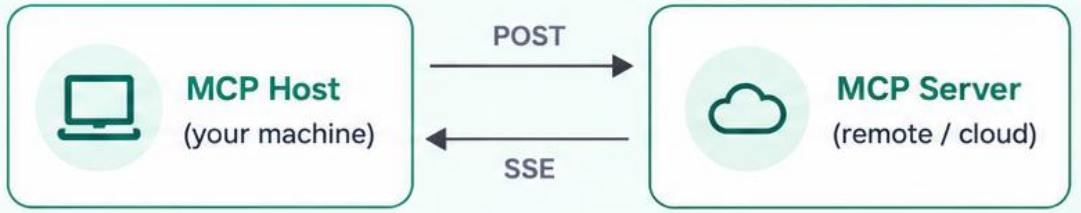
- Runs on your hardware (resource cost)
- Updates need a local reinstall
- One server process per user

https://github.com/modelcontextprotocol/servers/tree/main/src/filesystem

Remote MCP Server

Streamable HTTP transport

Hosted on the internet and serves many clients



PROS

- ✓ Auto-updates apply to all clients
- ✓ One server serves many users
- ✓ Offloads heavy compute to the cloud

CONS

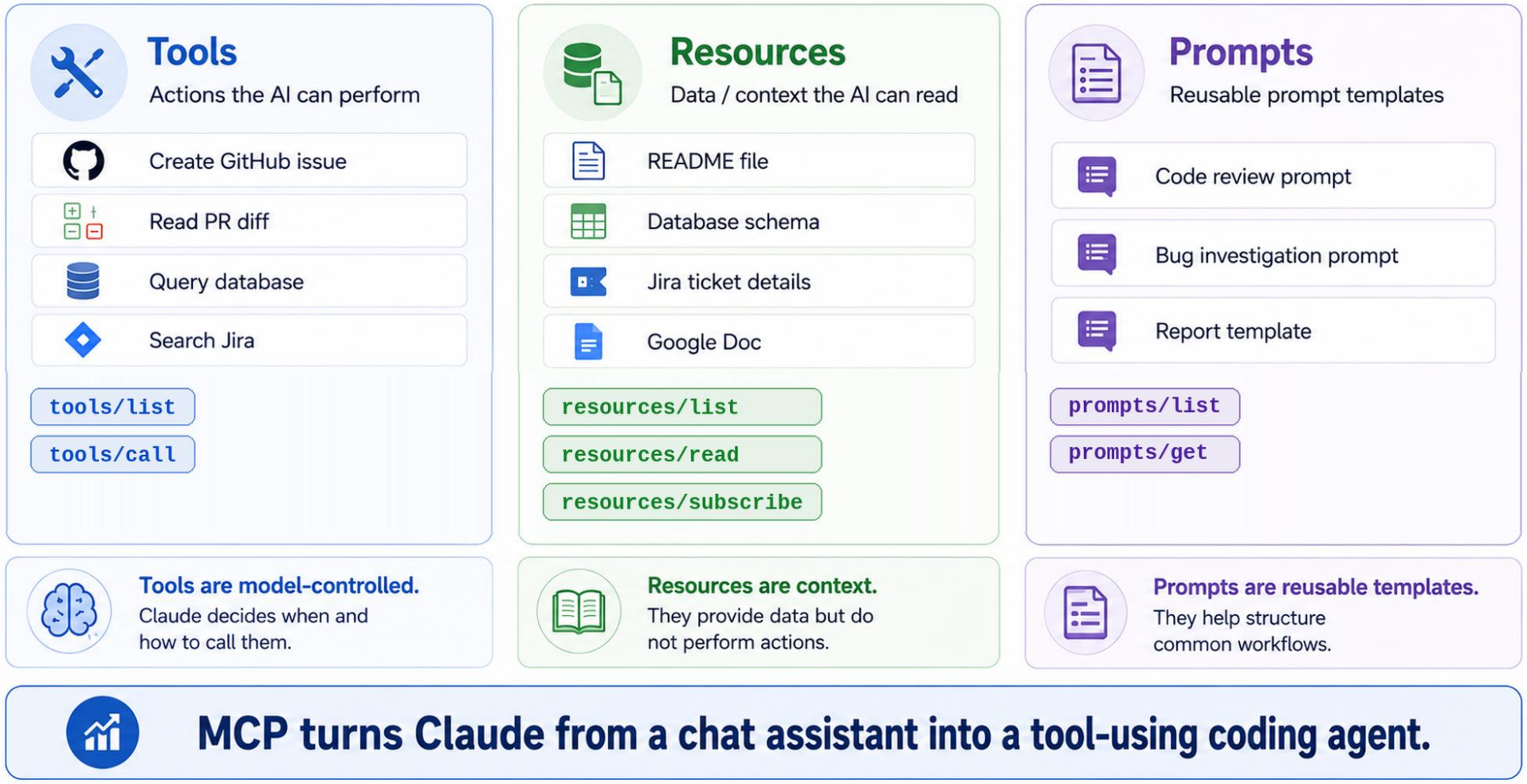
- Slight network latency
- Needs internet connectivity
- Auth via OAuth / API keys

https://developers.google.com/workspace/guides/configure-mcp-servers

Current standard transports: stdio (local) and Streamable HTTP (remote).

The earlier HTTP+SSE transport is deprecated and kept only for backward compatibility.

Primitives MCP Server Exposes to MCP Client



MCP Servers Requests from Client

Sampling
Ask the AI for its intelligence

The server asks the client's LLM to generate text. The client keeps control of model access.

"50 files changed — ask the AI to write a commit message."

`sampling/createMessage`

Roots
Ask which folders it may use

The server asks the client which directories it is allowed to read from and write to.

"Where am I allowed to work?" → client returns allowed paths.

`roots/list`

Elicitation
Ask the human for input

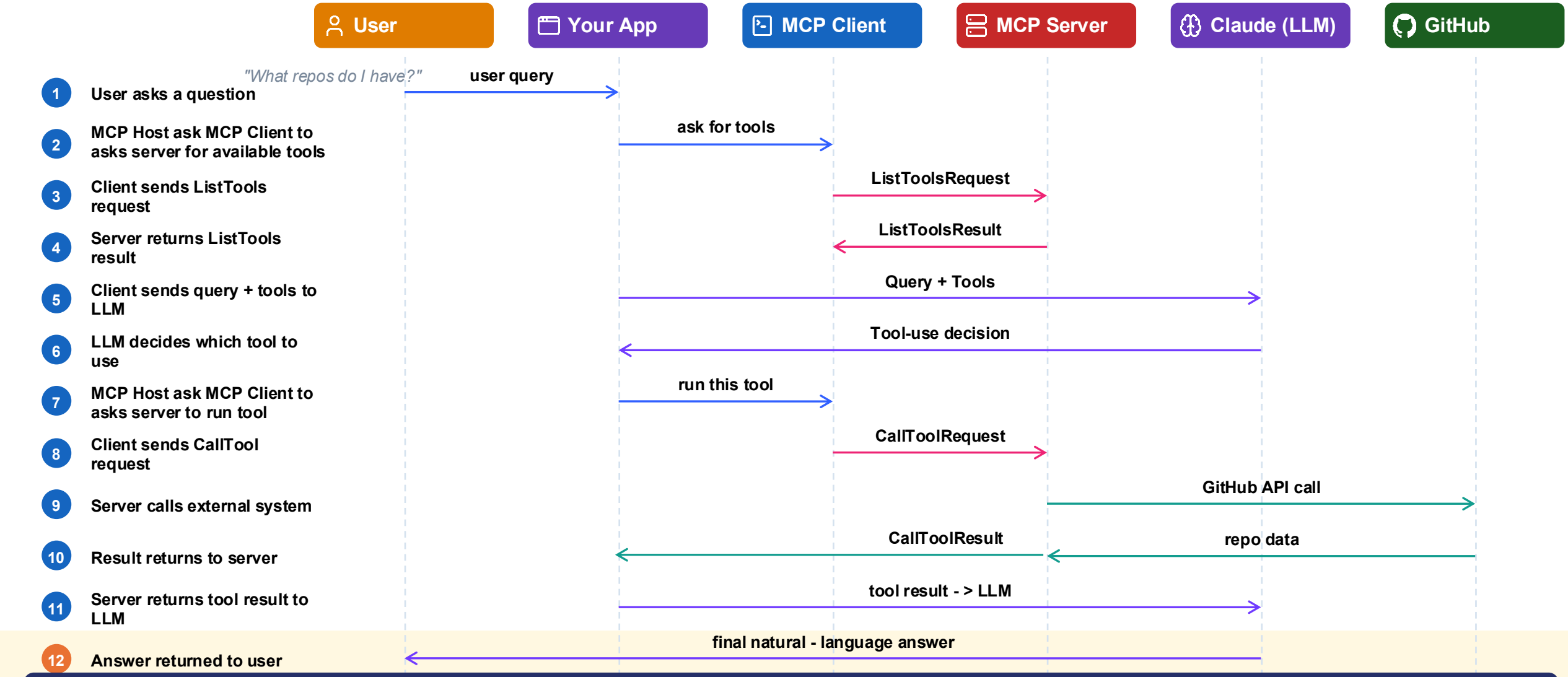
The server asks the client to collect a specific piece of input directly from the user.

"I need the DB password" → client asks the user for it.

`elicitation/create`

MCP lets the server ask the client for help — model sampling, file roots, and user input.

MCP Client-Server Communication



> Step 12: Answer returned to user

MCP Communication Life Cycle

1. INITIALIZATION PHASE

MCP Client → MCP Server

Client → Server: "initialize"
protocolVersion, capabilities (tools, examples), clientInfo (name, version)
Method: initialize

Server → Client: "initialize response"
protocolVersion, capabilities (logging, prompts, resources, tools), serverInfo (name, version)
Method: (initialize response)

Client → Server: "notifications/initialized"
client confirms readiness for normal operations
Method: notifications/initialized

- Negotiate protocol version
- Exchange capabilities
- Share client/server metadata
- Establish session readiness

Handshake & capability negotiation

2. OPERATION PHASE (METHODS)

MCP Client ↔ MCP Server

A. Tools

	Method (Client → Server)	Purpose
→	tools/list	Get list of available tools
→	tools/call	Call a tool with arguments
←	(Result)	Tool execution result

B. Resources

	Method (Client → Server / Server → Client)	Purpose
→	resources/list	Get list of available resources
→	resources/subscribe	Subscribe to resource updates
→	resources/read	Read resource content
→	(Server → Client) notifications/resources/updated	Notify resource has been updated
→	resources/unsubscribe	Unsubscribe from resource updates
←	(Result)	Response / confirmation

C. Prompts

	Method (Client → Server)	Purpose
→	prompts/list	Get list of available prompts
→	prompts/get	Get prompt template with arguments
←	(Result)	Prompt messages (template filled)

- Discover tools, resources, and prompts
- Execute tools and access data
- Receive live updates and notifications

Normal work and data exchange

3. SHUTDOWN PHASE

MCP Client → MCP Server

Client → Server: "shutdown"
request graceful session termination
Method: shutdown

Server → Client: "shutdown response"
result: null
Method: (shutdown response)

Client → Server: "notifications/exit"
final exit notification, no response expected
Method: notifications/exit

- Finish or cancel ongoing work gracefully
- Free resources
- Close the connection cleanly

Graceful termination

1 Initialize → 2 Operate → 3 Shutdown

All communication uses structured JSON-RPC messages.

Skills vs MCP

SKILLS *you already know these*

The HOW — procedural knowledge

- A folder with a `SKILL.md` — instructions, plus optional scripts & files
- Progressive disclosure: name → full guide → extra files, only as needed
- Teaches Claude how to do a task your way: formatting, workflows, conventions
- On its own it can guide, but can't reach external/live systems

Portable: runs in Claude Code, the API & other agent platforms (open standard).

Claude's brain · the recipe book

MCP *(Model Context Protocol)*

The WHAT — access to tools & data

- Open protocol (JSON-RPC) linking Claude to apps, data & services
- Discovers tools at runtime (`tools/list`), then calls them (`tools/call`)
- Brings live data, auth & real actions — GitHub, databases, Slack…
- Gives access, not the method — doesn't teach the how

Myth-buster: early setups dumped all tool schemas upfront; today's tool-search loads them on demand.

Claude's hands · the toolbox

Use them together

MCP gives Claude the tools — Skills teach Claude how to use them well.

Example: MCP pulls build data from GitHub, then a Skill turns it into your standard trend report.

OWASP MCP Top 10

1 Token Mismanagement & Secret Exposure



Hard-coded or long-lived keys and tokens can leak through logs or context, giving attackers access to connected systems.

MCP01:2025

2 Privilege Escalation via Scope Creep



Permissions that start too broad or quietly grow over time let agents — or attackers — do far more than intended.

MCP02:2025

3 Tool Poisoning



Malicious instructions hidden in a tool's description, schema, or output can trick the model into harmful actions.

MCP03:2025

4 Software Supply Chain Attacks & Dependency Tampering



Compromised packages, connectors, or dependencies can introduce backdoors or silently change how agents behave.

MCP04:2025

5 Command Injection & Execution



Untrusted input fed into shell commands, scripts, or code can let attackers run arbitrary commands on the host.

MCP05:2025

6 Prompt Injection via Contextual Payloads



Malicious text smuggled into prompts or fetched data can override instructions and make the model misbehave.

MCP06:2025

7 Insufficient Authentication & Authorization



Weak or missing identity and access checks let the wrong users or services reach tools, data, and actions.

MCP07:2025

8 Lack of Audit and Telemetry



Without detailed, tamper-proof logs, attacks and abnormal behavior are hard to detect, investigate, or stop.

MCP08:2025

9 Shadow MCP Servers



Unapproved, unmonitored MCP servers running outside official oversight quietly expand the attack surface.

MCP09:2025

10 Context Injection & Over-Sharing



Shared or persistent context windows can leak sensitive data across users, tasks, or sessions.

MCP10:2025

What you grant a connected MCP server

- **Files, APIs, databases & cloud services** it is wired to (GitHub, Slack, Drive…).
- **Local (stdio) servers inherit your OS user** — your whole file system & OS.
- **Remote (HTTP) servers use stored credentials** — PATs or OAuth tokens.
- **Every server you connect** widens the trust boundary and attack surface.

Developer steps to secure its use

- **Apply least privilege; prefer read-only** access, especially for demos.
- **Review what each server can actually do** before you connect it.
- **OAuth 2.1 + PKCE, short-lived scoped tokens** — no static keys.
- **Keep secrets out of** plaintext config, logs & version control.
- **Treat tool output as untrusted;** sandbox servers & generated code.
- **Human-approve risky actions;** inventory, log & monitor everything.

Sources: MCP spec (modelcontextprotocol.io), OWASP Gen AI Security Project & CoSAI, 2026

Q/A



Thank you!